

1. Цель работы

Целью задания является реализация и анализ базового цикла решения задачи регрессии на табличных данных. В процессе выполнения необходимо:

- реализовать процедуру обучения и оценки модели без утечек данных;
- рассчитать и интерпретировать показатели качества MAE, RMSE, R^2 (при необходимости — RMSLE);
- исследовать влияние выбора функции потерь на характер и распределение ошибок модели.

2. Выбор датасета

Для анализа выбираем датасет raw_winequality_red из набора mnemoraorg/wine-quality-6k4

3. Подготовка данных

3.1. Загрузим необходимые библиотеки и датасет

```
%pip install -q datasets seaborn pandas matplotlib scikit-learn numpy
```

```
from datasets import load_dataset
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import sys, os, warnings

# Подавление предупреждений (чтобы не засоряли вывод)
for warn in [UserWarning, FutureWarning]:
    warnings.filterwarnings("ignore", category=warn)
```

```
dataset_dict = load_dataset("mnemoraorg/wine-quality-6k4", data_files="raw_winequality_red.csv", sep=';')

my_dataframe = dataset_dict["train"].to_pandas()

# Проверим размер и список колонок.
print("\nРазмер таблицы (строки, колонки):", my_dataframe.shape)
print("Колонки:", sorted(my_dataframe.columns.tolist()))
print("\nПервые строки:")
my_dataframe.head()
```

README.md: 100% 956/956 [00:00<00:00, 47.1kB/s]

raw_winequality_red.csv: 84.2k/? [00:00<00:00, 1.41MB/s]

Generating train split: 1599/0 [00:00<00:00, 17743.09 examples/s]

Размер таблицы (строки, колонки): (1599, 12)

Колонки: ['alcohol', 'chlorides', 'citric acid', 'density', 'fixed acidity', 'free sulfur dioxide', 'pH', 'quality', 'residual sugar', 'sulfates', 'sulfur dioxide', 'total sulfur dioxide']

Первые строки:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulfates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6

Next steps: [New interactive sheet](#)

3.2. Разбиваем набор на обучающую, валидационную и тестовую части в соотношении 60 / 20 / 20

```
# =====
# Удаляем дубликаты и пересобираем сплиты
# =====

from sklearn.model_selection import train_test_split

n_before = len(my_dataframe)
```

```

n_duplicates = my_dataframe.duplicated().sum()

print(f"Размер до удаления дубликатов: {my_dataframe.shape}")
print(f"Число полностью дублирующихся строк: {n_duplicates}")

my_dataframe = my_dataframe.drop_duplicates().reset_index(drop=True)

n_after = len(my_dataframe)
print(f"Размер после удаления дубликатов: {my_dataframe.shape}")
print(f"Фактически удалено строк: {n_before - n_after}")

target_col = "quality"

drop_cols = [target_col]
feature_cols = [c for c in my_dataframe.columns if c not in drop_cols]
target_vector = my_dataframe[target_col].copy()

X = my_dataframe[feature_cols]
y = my_dataframe[target_col]

print("\nЧисло признаков:", X.shape[1])
print("Список признаков:", feature_cols)

# 20% в тест
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=111,
)

# Из оставшихся 80% -> 60% train, 20% val
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=111,
) # 0.25 * 0.8 = 0.2

print("\nTrain:", X_train.shape, y_train.shape)
print("Val: ", X_val.shape, y_val.shape)
print("Test: ", X_test.shape, y_test.shape)

```

```

Размер до удаления дубликатов: (1599, 12)
Число полностью дублирующихся строк: 240
Размер после удаления дубликатов: (1359, 12)
Фактически удалено строк: 240

Число признаков: 11
Список признаков: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide',

Train: (815, 11) (815,)
Val:   (272, 11) (272,)
Test:  (272, 11) (272,)

```

Разведочный анализ:

```

cols_for_stats = feature_cols + [target_col]
print(f"\nРаспределение ({target_col}): \n")
print(my_dataframe[target_col].value_counts().round(3))

print(y_test.value_counts().round(3))

my_dataframe[cols_for_stats].describe().T

```

Распределение (quality):

```
quality
5    577
6    535
7    167
4     53
8     17
3     10
Name: count, dtype: int64

quality
6    120
5    114
7     29
4      7
3      2
Name: count, dtype: int64
```

	count	mean	std	min	25%	50%	75%	max
fixed acidity	1359.0	8.310596	1.736990	4.60000	7.1000	7.9000	9.20000	15.90000
volatile acidity	1359.0	0.529478	0.183031	0.12000	0.3900	0.5200	0.64000	1.58000
citric acid	1359.0	0.272333	0.195537	0.00000	0.0900	0.2600	0.43000	1.00000
residual sugar	1359.0	2.523400	1.352314	0.90000	1.9000	2.2000	2.60000	15.50000
chlorides	1359.0	0.088124	0.049377	0.01200	0.0700	0.0790	0.09100	0.61100
free sulfur dioxide	1359.0	15.893304	10.447270	1.00000	7.0000	14.0000	21.00000	72.00000
total sulfur dioxide	1359.0	46.825975	33.408946	6.00000	22.0000	38.0000	63.00000	289.00000
density	1359.0	0.996709	0.001869	0.99007	0.9956	0.9967	0.99782	1.00369
pH	1359.0	3.309787	0.155036	2.74000	3.2100	3.3100	3.40000	4.01000
sulphates	1359.0	0.658705	0.170667	0.33000	0.5500	0.6200	0.73000	2.00000
alcohol	1359.0	10.432315	1.082065	8.40000	9.5000	10.2000	11.10000	14.90000
quality	1359.0	5.623252	0.823578	3.00000	5.0000	6.0000	6.00000	8.00000

3.3. Предобработка данных

```
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer

numeric_transformer = StandardScaler()

features_numeric_columns = (
    my_dataframe[feature_cols]
    .select_dtypes(include="number")
    .columns
    .tolist()
)

# Итоговый трансформер
preprocessor = ColumnTransformer(
    transformers=[
        ("scale_numeric", numeric_transformer, features_numeric_columns),
    ],
    remainder="drop"
)

transformed_train = preprocessor.fit_transform(X_train) # обучили трансформеры на train и применили
print("\nФорма данных до трансформации:", X_train.shape)
print("Форма данных после трансформации:", transformed_train.shape)
```

```
Форма данных до трансформации: (815, 11)
Форма данных после трансформации: (815, 11)
```

4. Построение бейзлайн-моделей

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

results = {}

def log_results(model_name: str, split: str, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
```

```

mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)

if model_name not in results:
    results[model_name] = {}

results[model_name][split] = {
    "mse": mse,
    "rmse": rmse,
    "mae": mae,
    "r2": r2,
}

def print_metrics(split_name: str, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"=== {split_name} ===")
    print(f"MSE : {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"MAE : {mae:.2f}")
    print(f"R2  : {r2:.4f}")
    print()

# =====
# Бейзлайн: предсказываем качество по train
# =====

baseline_name = "Baseline: mean(quality)"

# среднее только по train
y_train_mean = y_train.mean()
y_train_median = y_train.median()
print(f"Среднее {target_col} на train: {y_train_mean:.2f}")

# предсказания: одно и то же число для всех объектов
y_train_pred = np.full_like(y_train, fill_value=y_train_mean, dtype=float)
y_val_pred = np.full_like(y_val, fill_value=y_train_mean, dtype=float)
#y_test_pred = np.full_like(y_test, fill_value=y_train_mean, dtype=float)

# логируем результаты
log_results(baseline_name, "train", y_train, y_train_pred)
log_results(baseline_name, "val", y_val, y_val_pred)
#log_results(baseline_name, "test", y_test, y_test_pred)

# печатаем метрики
print("Модель:", baseline_name)
print_metrics("Train", y_train, y_train_pred)
print_metrics("Val", y_val, y_val_pred)
#print_metrics("Test", y_test, y_test_pred)

baseline_name = "Baseline: median(quality)"

# предсказания: одно и то же число для всех объектов
y_train_pred = np.full_like(y_train, fill_value=y_train_median, dtype=float)
y_val_pred = np.full_like(y_val, fill_value=y_train_median, dtype=float)
#y_test_pred = np.full_like(y_test, fill_value=y_train_median, dtype=float)

# логируем результаты
log_results(baseline_name, "train", y_train, y_train_pred)
log_results(baseline_name, "val", y_val, y_val_pred)
#log_results(baseline_name, "test", y_test, y_test_pred)

# печатаем метрики
print(f"Среднее {target_col} на train: {y_train_median:.2f}")
print("Модель:", baseline_name)
print_metrics("Train", y_train, y_train_pred)
print_metrics("Val", y_val, y_val_pred)
#print_metrics("Test", y_test, y_test_pred)

```

```

Среднее quality на train: 5.63
Модель: Baseline: mean(quality)
=== Train ===
MSE : 0.75
RMSE: 0.86
MAE : 0.72
R2  : 0.0000

=== Val ===
MSE : 0.60

```

```

RMSE: 0.78
MAE : 0.66
R2  : -0.0004

Среднее quality на train: 6.00
Модель: Baseline: median(quality)
=== Train ===
MSE : 0.88
RMSE: 0.94
MAE : 0.71
R2  : -0.1840

=== Val ===
MSE : 0.75
RMSE: 0.87
MAE : 0.64
R2  : -0.2465

```

Получаем лучшее качество на предсказании средним значением, тк наше распределение не симметричное (предсказание медианой дало отрицательный результат для R2). Медианная модель делает меньше "средних" ошибок (лучше MAE), но допускает несколько больших ошибок (хуже MSE/RMSE).

4. Обучение линейной модели с различными функциями потерь

4.1. Модель LinearRegression с MSE

```

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import SGDRegressor

linreg_sgd_name = "Linear Regression (SGD, MSE)"

linreg_sgd_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SGDRegressor(
        loss='squared_error', # MSE (значение по умолчанию)
        penalty=None,         # без регуляризации
        max_iter=1000,        # максимальное количество эпох
        tol=1e-3,             # критерий остановки
        random_state=111
    ))
])

# обучение на train
linreg_sgd_pipeline.fit(X_train, y_train)

# предсказания
y_train_pred = linreg_sgd_pipeline.predict(X_train)
y_val_pred   = linreg_sgd_pipeline.predict(X_val)
y_test_pred  = linreg_sgd_pipeline.predict(X_test)

# логируем результаты
log_results(linreg_sgd_name, "train", y_train, y_train_pred)
log_results(linreg_sgd_name, "val", y_val, y_val_pred)
log_results(linreg_sgd_name, "test", y_test, y_test_pred)

print("Модель:", linreg_sgd_name)
print_metrics("Train", y_train, y_train_pred)
print_metrics("Val", y_val, y_val_pred)
print_metrics("Test", y_test, y_test_pred)

```

```

Модель: Linear Regression (SGD, MSE)
=== Train ===
MSE : 0.45
RMSE: 0.67
MAE : 0.51
R2  : 0.4031

=== Val ===
MSE : 0.43
RMSE: 0.65
MAE : 0.52
R2  : 0.2949

```

Сравним результат с бейзлайнами.

Величина R2 ближе к 1, значит модель лучше обычного предсказания среднего. Значения абсолютной и квадратичной ошибок также снизились.

4.2. Построим диаграмму соответствия и график остатков.

```
import matplotlib.pyplot as plt
import numpy as np

# Предсказания
y_pred = y_val_pred
y_true = y_val

plt.figure()
plt.scatter(y_pred, y_true)

# линия y = x
min_val = min(y_pred.min(), y_true.min())
max_val = max(y_pred.max(), y_true.max())
plt.plot([min_val, max_val], [min_val, max_val])

plt.xlabel("Predicted ( $\hat{y}$ )")
plt.ylabel("Actual (y)")
plt.title("Predicted vs Actual")
print("Диаграмма соответствия")
plt.show()

residuals = y_true - y_pred

plt.figure()
plt.scatter(y_pred, residuals)

plt.axhline(y=0)
plt.xlabel("Predicted ( $\hat{y}$ )")
plt.ylabel("Residuals (y -  $\hat{y}$ )")
plt.title("Residuals vs Predicted")
print("График остатков")
plt.show()
```

Диаграмма соответствия

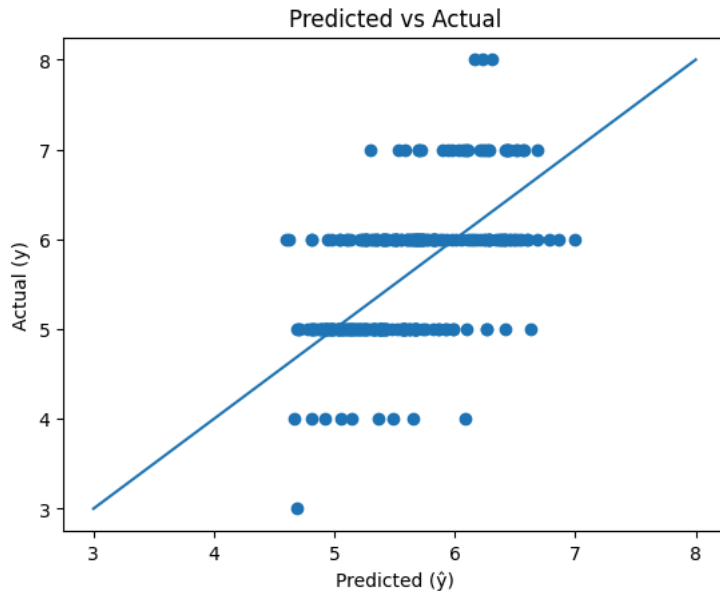
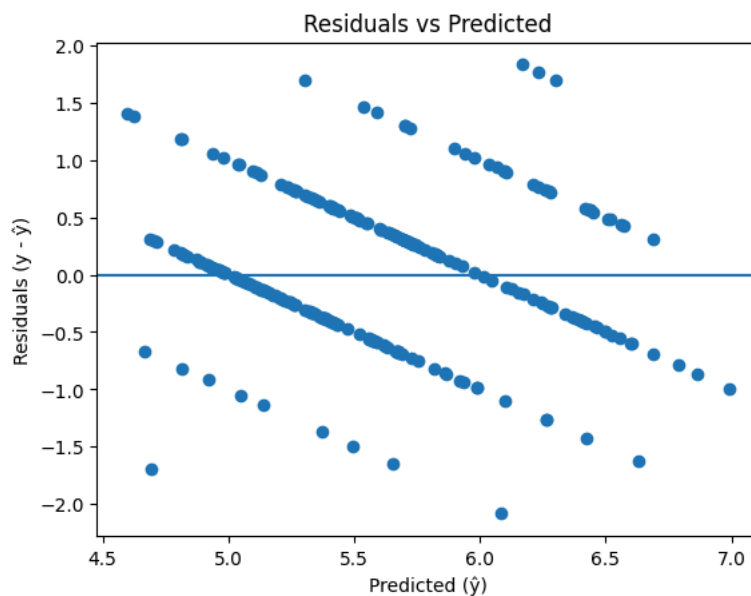


График остатков



4.3. Модель LinearRegression с MAE

```
linreg_mae_name = "Linear Regression (MAE)"

linreg_mae_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SGDRegressor(
        loss='epsilon_insensitive',
        epsilon=0.0,
        penalty=None,           # без регуляризации
        max_iter=1000,
        random_state=111
    ))
])

# обучение на train
linreg_mae_pipeline.fit(X_train, y_train)

# предсказания
y_train_pred = linreg_mae_pipeline.predict(X_train)
y_val_pred   = linreg_mae_pipeline.predict(X_val)
#y_test_pred = linreg_mae_pipeline.predict(X_test)

# логируем результаты
log_results(linreg_mae_name, "train", y_train, y_train_pred)
log_results(linreg_mae_name, "val", y_val, y_val_pred)
#log_results(linreg_mae_name, "test", y_test, y_test_pred)

print("Модель:", linreg_mae_name)
print_metrics("Train", y_train, y_train_pred)
```

```
print_metrics("Val", y_val, y_val_pred)
#print_metrics("Test", y_test, y_test_pred)
```

```
Модель: Linear Regression (MAE)
=== Train ===
MSE : 0.45
RMSE: 0.67
MAE : 0.50
R2  : 0.3903

=== Val ===
MSE : 0.44
RMSE: 0.66
MAE : 0.51
R2  : 0.2740
```

4.4. Выбираем лучшую модель

```
%pip install -q colorama
```

```
from rich.console import Console
from rich.table import Table
from rich import box
import pandas as pd

console = Console()

def print_colored_results_table_rich(results_dict):
    """
    Печатает результаты в виде цветной таблицы с использованием rich
    """
    # Создаем таблицу
    table = Table(
        title="📊 Результаты моделей (Validation)",
        box=box.ROUNDED,
        show_header=True,
    )

    # Добавляем колонки (убираем колонку "Сплит", так как теперь только Val)
    table.add_column("Модель", style="white", no_wrap=True)
    table.add_column("MSE", justify="right", style="white")
    table.add_column("RMSE", justify="right", style="white")
    table.add_column("MAE", justify="right", style="white")
    table.add_column("R²", justify="right", style="white")

    # Фильтруем только validation сплиты
    val_metrics = []
    val_data = [] # Для хранения данных для таблицы

    for model_name, splits in results_dict.items():
        for split_name, metrics in splits.items():
            # Проверяем разные варианты написания "val"
            if split_name.lower() in ['val', 'validation', 'valid', 'validate']:
                val_metrics.append(metrics)
                val_data.append({
                    'model': model_name,
                    'metrics': metrics
                })

    # Проверяем, есть ли validation данные
    if not val_metrics:
        console.print("[bold red]Ошибка: Не найдены validation данные![bold red]")
        console.print("Доступные сплиты:", [s for splits in results_dict.values() for s in splits.keys()])
        return

    # Находим лучшие значения только среди validation данных
    best_mse = min(m['mse'] for m in val_metrics)
    best_rmse = min(m['rmse'] for m in val_metrics)
    best_mae = min(m['mae'] for m in val_metrics)
    best_r2 = max(m['r2'] for m in val_metrics)

    worst_mse = max(m['mse'] for m in val_metrics)
    worst_rmse = max(m['rmse'] for m in val_metrics)
    worst_mae = max(m['mae'] for m in val_metrics)
    worst_r2 = min(m['r2'] for m in val_metrics)

    # Заполняем таблицу только validation данными
    for item in val_data:
        model_name = item['model']
        metrics = item['metrics']
```



```

# Определяем цвета для каждого значения
# MSE: меньше = лучше
mse_style = "bold green" if metrics['mse'] == best_mse else "bold red" if metrics['mse'] == worst_mse else "white"
mae_style = "bold green" if metrics['mae'] == best_mae else "bold red" if metrics['mae'] == worst_mae else "white"
r2_style = "bold green" if metrics['r2'] == best_r2 else "bold red" if metrics['r2'] == worst_r2 else "white"
rmse_style = "bold green" if metrics['rmse'] == best_rmse else "bold red" if metrics['rmse'] == worst_rmse else "white"

table.add_row(
    model_name,
    f"[{mse_style}]{metrics['mse']:.4f}[/{mse_style}]",
    f"[{rmse_style}]{metrics['rmse']:.4f}[/{rmse_style}]",
    f"[{mae_style}]{metrics['mae']:.4f}[/{mae_style}]",
    f"[{r2_style}]{metrics['r2']:.4f}[/{r2_style}]"
)

# Печатаем таблицу
console.print()
console.print(table)
console.print()

# Легенда
console.print("[bold green]Зеленый[/bold green] - лучшее значение на validation")
console.print("[red]Красный[/red] - худшее значение на validation")

# Используем rich версию (работает в Colab)
print_colored_results_table_rich(results)

```

 Результаты моделей (Validation)

Модель	MSE	RMSE	MAE	R ²
Baseline: mean(quality)	0.6049	0.7778	0.6625	-0.0004
Baseline: median(quality)	0.7537	0.8681	0.6434	-0.2465
Linear Regression (SGD, MSE)	0.4264	0.6530	0.5155	0.2949
Linear Regression (MAE)	0.4390	0.6626	0.5142	0.2740

Зеленый - лучшее значение на validation
Красный - худшее значение на validation

Модель LinearRegression с MSE показала лучшие результаты на валидирующей выборке по 2 из 3 показателей. Возьмем ее в качестве лучшей. Хотя видно, что модель Linear Regression в целом плохо подходит для данной выборки (низкий R2).

5. Финальная оценка

```

# предсказания
y_train_pred = linreg_sgd_pipeline.predict(X_train)
y_test_pred = linreg_sgd_pipeline.predict(X_test)

# логируем результаты
log_results(linreg_sgd_name, "test", y_test, y_test_pred)

print("Модель:", linreg_sgd_name)
print_metrics("Train", y_train, y_train_pred)
print_metrics("Test", y_test, y_test_pred)

```

```

Модель: Linear Regression (SGD, MSE)
=== Train ===
MSE : 0.45
RMSE: 0.67
MAE : 0.51
R2  : 0.4031

=== Test ===
MSE : 0.41
RMSE: 0.64
MAE : 0.48
R2  : 0.2543

```

6. Попробуем другую функцию потерь (Huber)

```

linreg_hub_name = "Linear Regression (Huber)"

linreg_hub_pipeline = Pipeline([
    ("scaler", StandardScaler()),

```

```
( "model", SGDRegressor(  
    loss='huber',  
    epsilon=1.35,  
    penalty='l1', alpha=0.0001,  
    max_iter=1000,  
    random_state=111,  
  
    ))  
])  
  
# обучение на train
```